Parallel Computing Assignment 2

A*, concurrent sorted lists and parallel search
English edition: 24 October 2026 | Experiments: 16 October 2026
Repository: GitGud031005/parallel-computing-assignment

Part	Deliverable
Sequential A*	Heap priority; scalar distance/time and two-objective Pareto frontiers; four heuristic policies.
Concurrent sets	Coarse mutex and fine-grained lock coupling; insert, erase, contains and atomic pop-min.
Parallel A*	Coarse/fine priority lists, per-vertex label locks, reopen and outstanding-work accounting.
Correctness	Independent oracles, exhaustive small graphs, bounded histories, directed schedules and sanitizers.
Experiments	Real HCM OSM extract, exact quality checks, execution-time/scaling sweeps, records, summaries and figures.

The reference matrix contains 356 configurations and 2136 samples including warmups; 1780 samples are measured repetitions. All recorded samples passed quality/completeness checks. Optional contraction hierarchies are not implemented. The managed host does not substitute for experiments on the submitter's owned machines.

1. Road-network model

The HCM extract covers bbox 10.770,106.685,10.785,106.705 (Overpass snapshot 2026-10-16): 3106 nodes and 5350 directed edges, largest SCC 2904 nodes. Each segment has integer (distance_mm, estimated_time_ms). Spherical segment length is rounded up to millimeters; static road-class speeds give time rounded up to milliseconds. This is not observed or live traffic, and turn restrictions are not modeled.

OSM node IDs preserve connectivity and one-way/roundabout direction. GPS endpoints snap to the closest existing graph node; roads at different elevations are not joined by coordinate proximity. The extract is a small part of HCM, not the entire city. Data: © OpenStreetMap contributors, ODbL-1.0. https://www.openstreetmap.org/copyright

2. A*, dynamic/uncertain h and vector f

g is the path cost already incurred, h a lower bound on remaining cost, and f=g+h. Costs are finite nonnegative integers with checked arithmetic. Improved labels reopen a vertex; there is no permanent closed set. Each label holds immutable parent and edge IDs so later improvements cannot corrupt reconstructed paths, including parallel edges.

Policy	Definition used
zero	h=(0,0), a uniform-cost ordering control. Shared setup still performs landmark preprocessing.
landmark	Componentwise maximum of 0, d(L,t)-d(L,v), d(v,L)-d(t,L) over finite terms and two landmarks.
dynamic	epoch=ticket/period; scale landmark bound by 1, 1/4, 3/4, 1/2 cyclically and floor.
uncertain	Chosen estimate band [floor(L/2), ceil(3L/2)]; only the certified lower endpoint is materialized for ordering/pruning.

Two landmarks, forward/reverse shortest distances and two components require eight Dijkstra preprocessing runs. Directed triangle inequalities justify each finite lower-bound term; infinity is never subtracted. Tasks snapshot h when created; queued keys do not mutate. Every dynamic snapshot remains admissible, even across epochs. Edge costs remain static throughout a query.

The uncertain band is neither a probability model nor a confidence interval. Its upper endpoint is not a certified upper bound on true shortest cost and is not used to prune. If an estimate has no trustworthy lower bound, h=0 is safe. This does not solve stochastic or departure-time-dependent traffic routing.

For vector f, each vertex retains all nondominated g labels. Componentwise weak dominance uses <= in every component; equality keeps one representative. Lexicographic (f_distance,f_time,ticket) controls processing order only. Weighted sums and first-goal termination are not used. The synthetic vectors (2,18), (12,12), (18,2) test retention of an unsupported intermediate Pareto point.

The course supplies A* and asks open dynamic/uncertain/vector questions. ALT is an optional extension outside the supplied slides; the scale/band and Pareto model are explicit implementation choices, not formulas prescribed by the lecturer.

3. Sorted lists and parallel coordination

Structure	Synchronization / linearization
CoarseList	One mutex protects traversal and link changes. Insert/erase/pop linearize at the link mutation; contains at its protected membership observation.
FineList	Lock predecessor/current in head-to-tail order. Keep current locked while acquiring its successor. Insert/erase hold both nodes; contains also uses lock coupling.
Lifetime	No node pointers escape. Unlink with both locks; unlock the victim before deleting it while predecessor still excludes new readers. Snapshot/destruction require all workers joined.
Priority adapter	Immutable f snapshot, unique atomic ticket and shared label; atomic try_pop_min. Distinct tasks at one vertex do not collapse into a duplicate set key.

A thread-safe queue alone is insufficient for parallel A*. A per-vertex mutex makes dominance checking, invalidation and label insertion one transaction. Goal-frontier updates have a separate mutex. The atomic alive flag marks stale labels; redundant expansion of an invalidated label is safe. No vertex lock is held while operating on the queue; no global scheduler mutex encloses list traversal.

Outstanding work = queued + active + reserved-before-publication. Reserve before enqueue; complete the parent only after publishing all children. Pop does not reduce the count. A transition to zero proves no active parent can generate further work. An empty queue while a worker expands makes others wait, not terminate. A condition-variable version under the wait mutex prevents lost wakeups without serializing list operations.

The first popped goal can be worse than a path still in flight. Only a feasible goal weakly dominating g+h permits pruning; processing continues to quiescence. Nonnegative additive costs justify dominated-prefix elimination. Equality deduplication removes zero-cycle duplicates. One representative path is returned per distinct nondominated cost vector.

Label-budget or worker exceptions cancel, wake and join workers, then report failure; an incomplete frontier is never labeled exact. Both lists/solvers are blocking, with no general starvation-freedom promise from std::mutex. Work-item quiescence is a termination argument, not an independent linearizability proof.

4. Correctness evidence

Check	Oracle / coverage
Lists	std::set sequential specification; int64 extremes, insert winner, exact pop sets, mixed stress; real-time-compatible serializations of 12-operation histories.
Search	Exhaustive simple paths on small graphs; scalar Dijkstra; complete Pareto equality; all policy/objective/engine combinations, cycles, zero edges, disconnected graphs and source=target.
Directed schedule	Hold the cheap branch active while an expensive goal is popped and queue becomes empty; solver must wait and return cost 2 rather than 20. Inject exceptions/budget cancellation.
Python	Independent scalar heap Dijkstra and Pareto FIFO label correction, without C++ A*/ALT/lists. Replay every reported path by directed edge ID and exact cost sum.
CLI / tools	Endpoint modes, parser/overflow, OSM filters/direction/missing references; known median/IQR; reject corrupted paths and missing/duplicate/bad raw samples.

Default search uses 15 seeded random small graphs, stress 60; list histories use 20 seeds, stress 100. Bounded histories do not cover all executions. Root CTest also runs Ass1 for regressions. Debug, Release, ASan+UBSan, UBSan and TSan checks passed; list/search stress passed. LeakSanitizer was explicitly disabled on the restricted /proc/ptrace host, so no LSan leak-free claim is made.

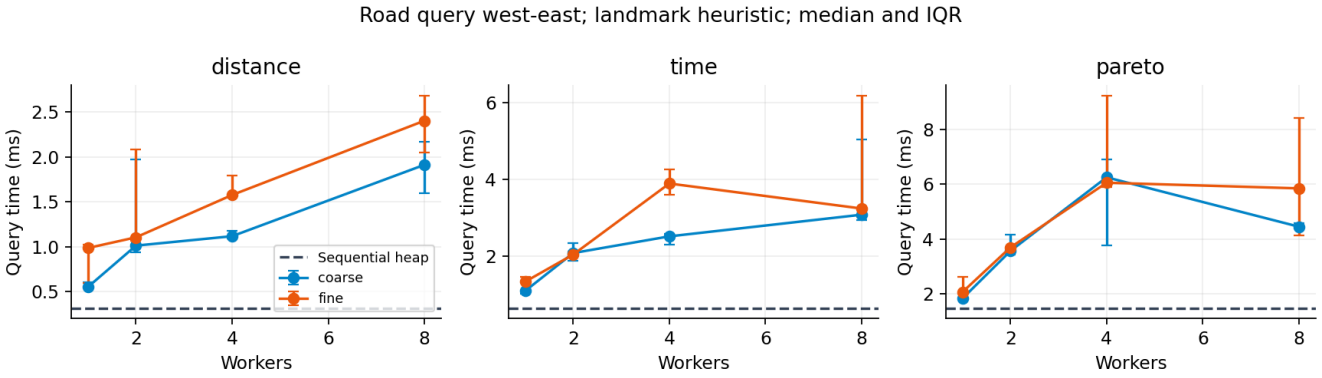
Every scalar sample must match the independent shortest cost and replay a valid path. Every Pareto sample must equal the complete oracle vector set, not merely its cardinality. Exactness is relative to the stated graph/cost model, not a real-world travel-time prediction. Finite tests/sanitizers are not exhaustive memory-model proofs.

Measured kernel/build SHA-256:
be4e1d367d9ed5b625172ad4189899b07672c8b877dfc13fe5347f0e3fa5727f
Graph SHA-256:
8a2a67e36e1b010f578b5ec438153c12c0a72c7ac4375ab88e0d06a951297e9a

Course mapping: Ch06 pp.4-24 for sorted sets, coarse/fine locking and deletion; Ch05 pp.15-19 for linearizability/progress; A* discussion pp.7-10 for g/h/f and open modifications; Ch04 for condition synchronization and termination concepts. See docs/course-review.md.

5. A* execution time and scalability

Host: AMD EPYC 9V74, Linux-6.18.44-x86_64-with-glibc2.39, nine visible/affinity CPUs, cpu.max=800000 100000; power mode uncontrolled. Each sample is a separate process. Query time includes state allocation, search, worker spawn/join and internal path reconstruction/validation. Loading and endpoint/landmark preprocessing are recorded separately; end-to-end timing includes them up to JSON output. Runner process wall time also includes startup and I/O.



Objective	Engine/P	Median ms	IQR ms	S vs seq
distance	sequential/1	0.318	0.316-0.327	1.000
distance	coarse/8	1.910	1.592-2.165	0.167
distance	fine/8	2.400	2.044-2.682	0.132
time	sequential/1	0.652	0.632-0.665	1.000
time	coarse/8	3.080	2.940-5.039	0.212
time	fine/8	3.240	3.020-6.185	0.201
pareto	sequential/1	1.484	1.390-1.769	1.000
pareto	coarse/8	4.450	4.371-4.591	0.333
pareto	fine/8	5.849	4.140-8.420	0.254

Figure/table: west-east query, landmark policy. Sequential-heap speedup and same-engine P=1 speedup are separate metrics. Startup, traversal and synchronization can dominate these small queries; parallel is slower here. This overhead explanation is a plausible interpretation, not hardware-counter evidence. Compare only matching query/objective/heuristic/graph/source.

6. Heuristics and concurrent-set workloads

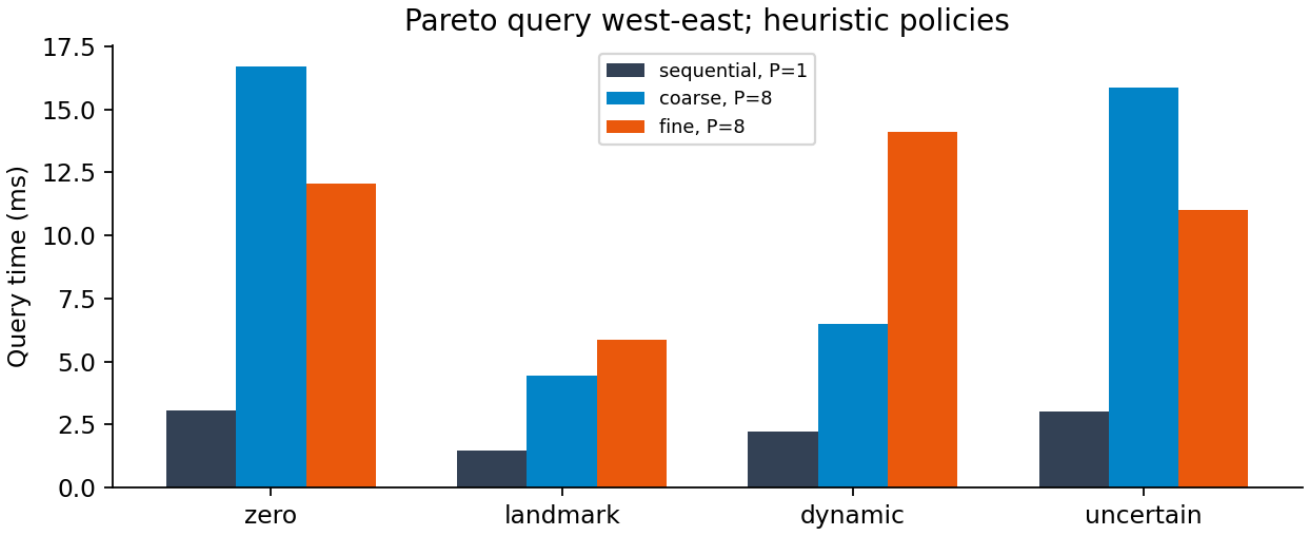


Figure 2. Pareto west-east query under four heuristic policies. A stronger bound can reduce search work without guaranteeing lower elapsed time. Dynamic snapshots and worker scheduling can change generated/expanded counts; complete output quality must remain exact.

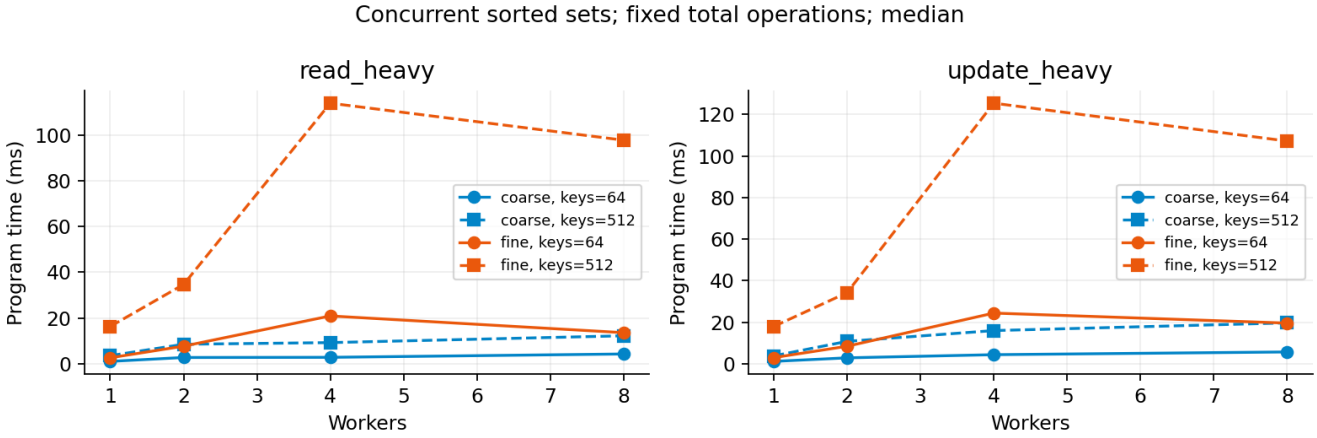


Figure 3. Fixed-total set benchmark: read-heavy contains/insert/erase=80/10/10; update-heavy=20/40/40. Key ranges 64 and 512, initially even keys. Variants share the configured seed/workload. Per-worker RNG means changing P changes stream partition; this does not imply the same full operation history.

Post-join sorted/unique/key-range/operation-count checks are invariants; semantic linearizability is tested separately using bounded histories. Fine locking permits concurrency on distinct list regions but adds many lock acquisitions during traversal. It may be slower than coarse locking in these workloads. Short runs have startup/scheduling noise; results do not establish universal rankings.

7. Route geometry and Pareto frontier

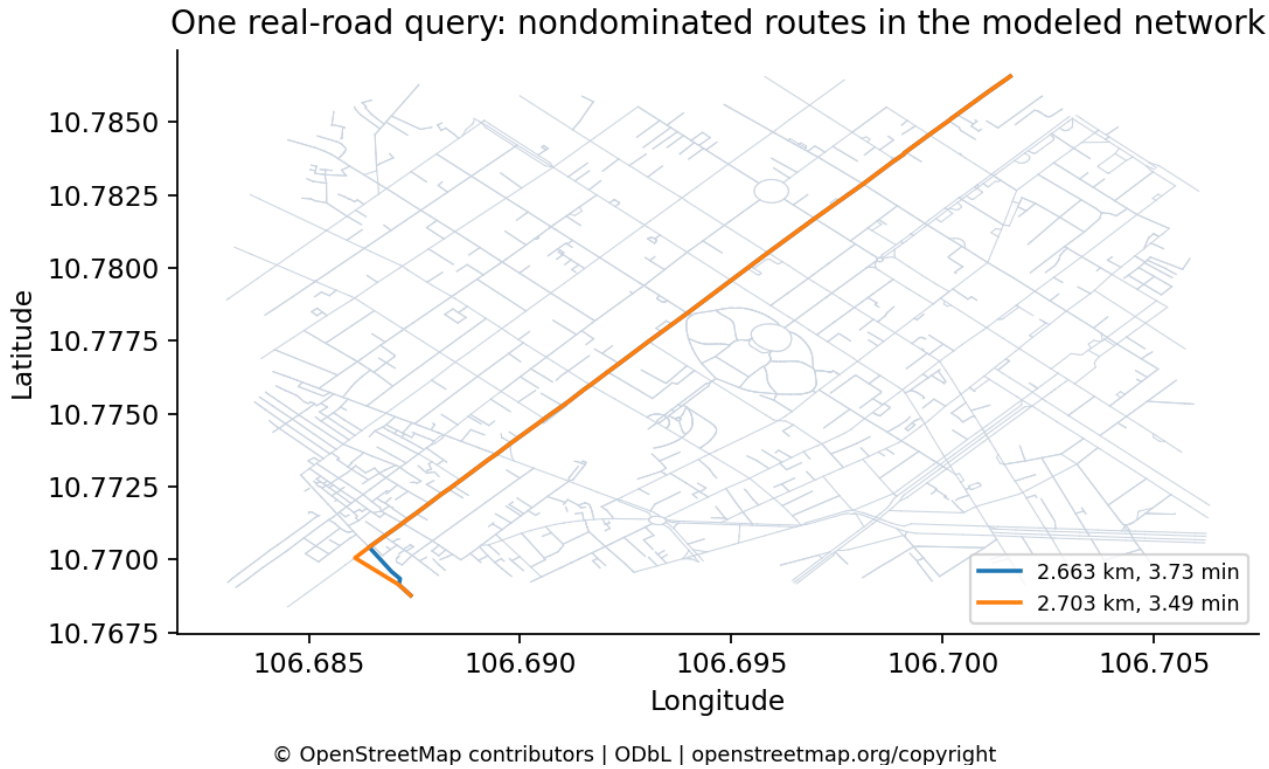


Figure 4. Nondominated routes for one real-road query. Paths follow directed edges from the extract; every edge ID and accumulated cost is independently replayed.

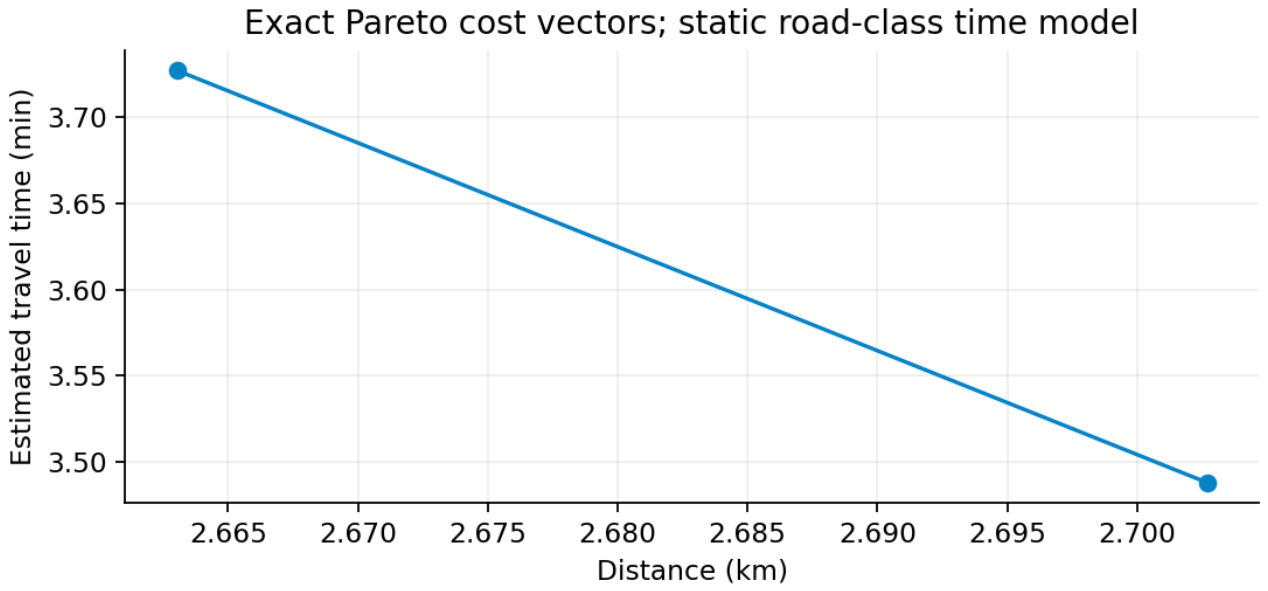


Figure 5. Exact nondominated cost vectors for that query in the static road-class-speed model. Routes can overlap geometrically while trading length against estimated travel time. The clipped extract omits turn restrictions. © OpenStreetMap contributors, ODbL-1.0: https://www.openstreetmap.org/copyright

8. Reproduction, limitations and references

README documents build/test, OSM download/conversion, index/OSM-ID/GPS endpoints, smoke/pilot/reference runs, summaries, figures, reports and packaging. Raw records retain warmups/errors; outliers are not removed by ranking. Median/inclusive IQR use measured repetitions only. Report generation rejects incomplete groups, quality errors and inconsistent source/check fingerprints.

Matrix: three HCM queries (west-east 382->85, south-north 612->2761, east-west 85->382), four policies and three objectives; sequential P=1, coarse/fine P=1/2/4/8. This gives 324 route cases plus 32 set cases, one warmup and five measured repetitions, timeout 60 seconds/sample, seed 20261016. No sanitizer is enabled in performance measurements. Pareto frontier sizes for these queries are 5, 2 and 5.

The implementation was also smoke-tested on a separate pinned West Oakland OSMnx fixture (129 nodes, 218 edges, SCC 80). Its results are not HCM measurements. The synthetic tradeoff.road fixture is authored test data, not a real road network.

Limitations: one managed platform, a small clipped road extract, static estimated speeds, no turn restrictions/live traffic, and no exhaustive model checking or scheduler-independent progress proof. Dynamic/uncertain refer to the defined heuristic models, not changing edge costs. Pareto label counts can grow substantially; reaching the budget reports failure instead of silently truncating exact results. The usual scalar A* complexity statement does not cover all reopening/Pareto variants.

This translation preserves the 16 October reference measurements and incorporates the later course-alignment clarifications; no new benchmark run is claimed. The statement specifies individual work, a ZIP containing PDF and Python/C++ source, and a deadline of 26 November 2026. Personal-machine measurements, submitter identification, LMS closing-time verification and submission are still required.

References

[1] Assignment2_v1_30092026.pdf, pp.1-2; A* discussion pp.4,7-10,12; osm-astar.ipynb. Supplied originals remain outside GitHub.
[2] Course Ch06 pp.4-24; Ch05 pp.15-19; Ch02a/Ch02b; Ch04 condition synchronization.
[3] Goldberg and Harrelson, Computing the Shortest Path: A* Search Meets Graph Theory, MSR-TR-2004-24 (2004). https://www.microsoft.com/en-us/research/publication/computing-the-shortest-path-a-search-meets-graph-theory/
[4] Maristany de las Casas et al., Targeted Multiobjective Dijkstra Algorithm (2021). https://arxiv.org/abs/2110.10978 . This implementation is label-correcting and does not claim to reproduce T-MDA.
[5] OpenStreetMap contributors, ODbL-1.0. https://www.openstreetmap.org/copyright . West Oakland fixture: gboeing/osmnx tests/input_data.
[6] C++20 working draft N4861, mutex/thread/atomic requirements. https://isocpp.org/files/papers/N4861.pdf